

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет прикладной информатики

Отчет по курсовой работе

Дисциплины Мобильная разработка (Android и IOS) на тему
«Разработка Android-приложения для обмена сообщениями»

Выполнил:

Студент группы K3441

Захарчук Александр Игоревич

Проверил:

Преподаватель ФПИН ИТМО

Шатинский Григорий Сергеевич

Санкт-Петербург, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 Общая архитектура приложения	5
1.1 Архитектурный подход MVVM	5
1.2 Структура приложения.....	5
2 Реализация навигации и жизненного цикла.....	7
2.1 Главная активность	7
2.2 Навигация	8
3 Экран профиля.....	9
4 Экран настроек	10
5 Загрузка данных из сети	11
6 Локальное хранилище данных.....	13
6.1 Общее описание	13
6.2 Использование Room.....	13
6.3 Работа с данными	14
7 Отображение списка сообщений.....	16
7.1 Использование RecyclerView	16
7.2 Специализированный адаптер	16
8 Фоновая синхронизация	18
9 Уведомления и разрешения	19
9.1 Уведомления.....	19
9.2 Разрешения	19
ЗАКЛЮЧЕНИЕ	21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	22

ВВЕДЕНИЕ

В настоящее время мобильные приложения являются неотъемлемой частью цифровой экосистемы. Особенно востребованными остаются приложения для обмена сообщениями, так как они обеспечивают оперативную коммуникацию между пользователями и служат основой для социальных, корпоративных и информационных сервисов. Платформа Android занимает лидирующие позиции на рынке мобильных операционных систем, что обуславливает актуальность разработки приложений именно под данную платформу.

Целью данной работы является разработка простого Android-приложения типа «Мессенджер», демонстрирующего применение современных подходов к архитектуре, управлению состоянием, работе с сетью и локальным хранилищем данных, а также реализации пользовательского интерфейса с использованием компонентов Material Design [6].

В ходе выполнения работы были поставлены и решены следующие задачи:

- разработка приложения с одной главной активностью и навигацией на основе Fragment;
- внедрение архитектуры MVVM для управления состоянием экранов;
- реализация загрузки данных из удаленного API;
- обеспечение офлайн-доступа к данным с использованием базы данных Room;
- улучшение пользовательского интерфейса и добавление фоновой синхронизации;
- реализация уведомлений и обработки разрешений.

В качестве языка программирования использован Kotlin [3]. Для реализации сетевого взаимодействия применялась библиотека Retrofit [4], для

асинхронных операций — Kotlin Coroutines, для локального хранения данных — Room, а для фоновых задач — WorkManager.

1 Общая архитектура приложения

Архитектура мобильного приложения определяет принципы организации кода, взаимодействия компонентов и распределения ответственности между слоями системы. Грамотно спроектированная архитектура упрощает сопровождение проекта, повышает его масштабируемость и снижает вероятность ошибок.

В рамках данной работы приложение построено с использованием архитектурного паттерна MVVM (Model–View–ViewModel). Данный подход рекомендован для Android-разработки и активно используется в современных проектах.

1.1 Архитектурный подход MVVM

Паттерн MVVM предполагает разделение приложения на три основных слоя:

- Model — слой данных, включающий модели, репозитории, сетевые и локальные источники данных;
- View — слой пользовательского интерфейса, представленный Activity и Fragment;
- ViewModel — промежуточный слой, отвечающий за хранение состояния и подготовку данных для отображения.

Основным преимуществом MVVM является отсутствие прямой зависимости View от Model. View взаимодействует только с ViewModel, а обновление интерфейса происходит реактивно за счет использования LiveData.

1.2 Структура приложения

Приложение содержит одну главную активность MainActivity, которая выступает в качестве точки входа. Все экраны приложения реализованы в виде фрагментов и размещаются внутри NavHostFragment.

Основные функциональные модули приложения:

- модуль пользовательского интерфейса (UI);

- модуль управления состоянием (ViewModel);
- модуль работы с данными (Repository, Room, Retrofit);
- модуль фоновых задач и уведомлений.

Общая архитектура приложения показана на рисунке 1.

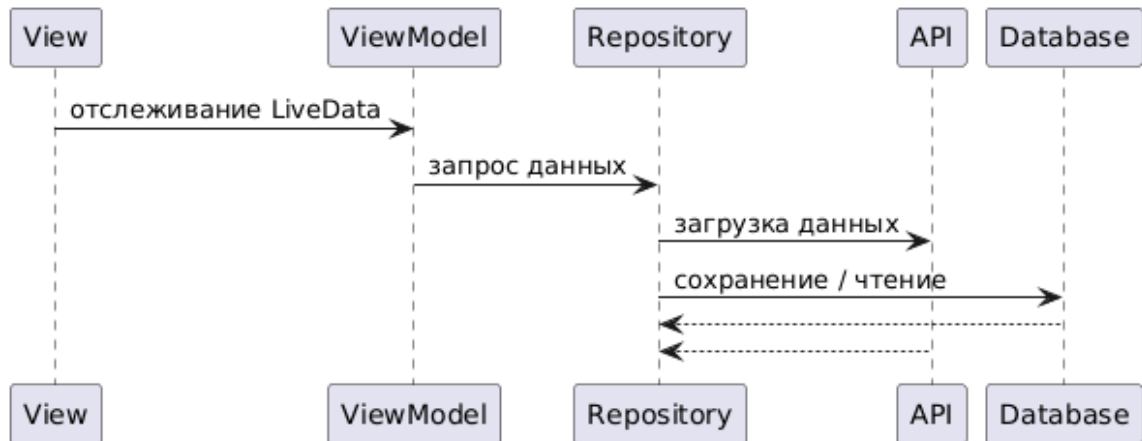


Рисунок 1 - Общая архитектура приложения

2 Реализация навигации и жизненного цикла

2.1 Главная активность

Главная активность `MainActivity` выполняет исключительно роль контейнера для фрагментов и инициализации навигации. Такой подход соответствует принципам разделения ответственности и позволяет избежать перегруженности активности бизнес-логикой.

В методе `onCreate()` производится:

- установка разметки активности;
- инициализация `NavHostFragment`;
- связывание `BottomNavigationView` с `NavController`.

Для целей отладки и анализа жизненного цикла в активности реализовано логирование методов `onCreate()` и `onDestroy()`.

Внешний вид главной активности показан на рисунке 2.

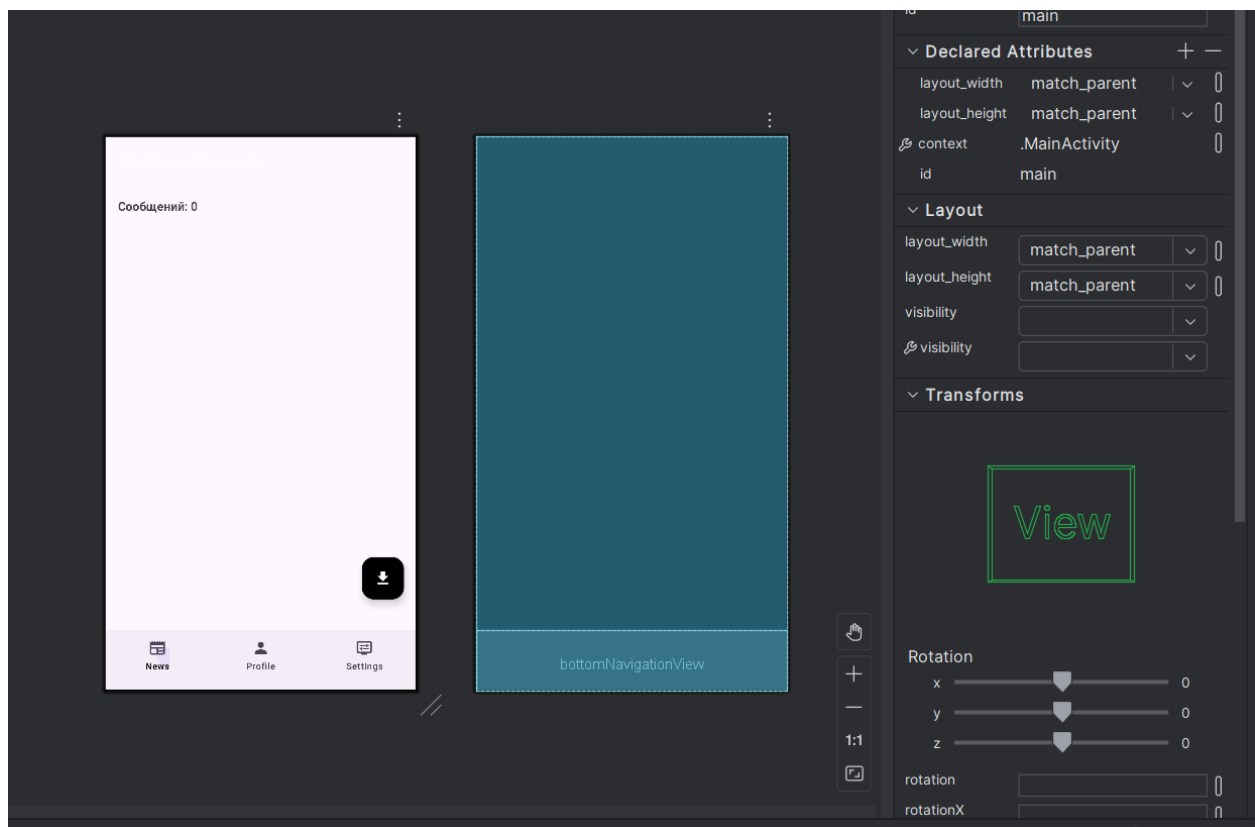


Рисунок 2 - Главная активность

2.2 Навигация

Для управления навигацией используется Android Navigation Component. Все экраны приложения описаны в Navigation Graph [2], где также задан стартовый экран — лента сообщений.

Использование Navigation Component позволяет:

- централизованно управлять навигацией;
- избежать ручного управления транзакциями фрагментов;
- обеспечить корректную работу кнопки «Назад».

Диаграмма навигации показана на рисунке 3.

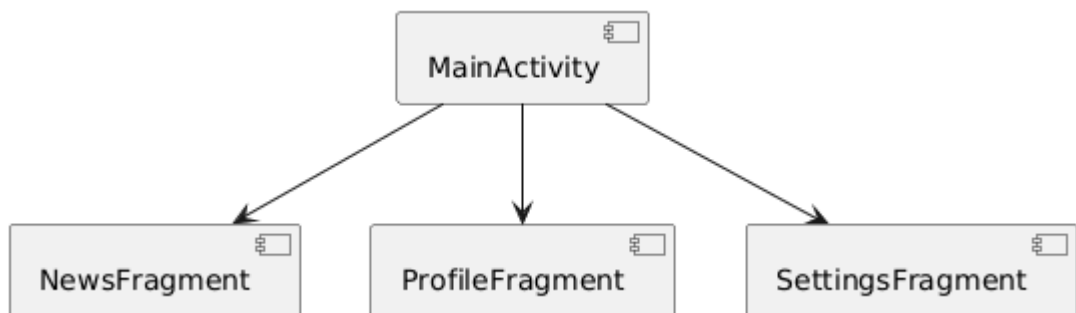


Рисунок 3 – Навигация

3 Экран профиля

Экран профиля предназначен для отображения и редактирования пользовательских данных, таких как имя и статус. Эти данные хранятся в ViewModel и доступны через LiveData.

ProfileViewModel отвечает за хранение состояния профиля. При изменении имени или статуса данные автоматически обновляются в пользовательском интерфейсе благодаря использованию LiveData.

При повороте экрана состояние не теряется, так как ViewModel сохраняется системой Android.

В жизненном цикле ViewModel также реализовано логирование этапов инициализации и уничтожения.

Внешний вид экрана профиля показан на рисунке 4.

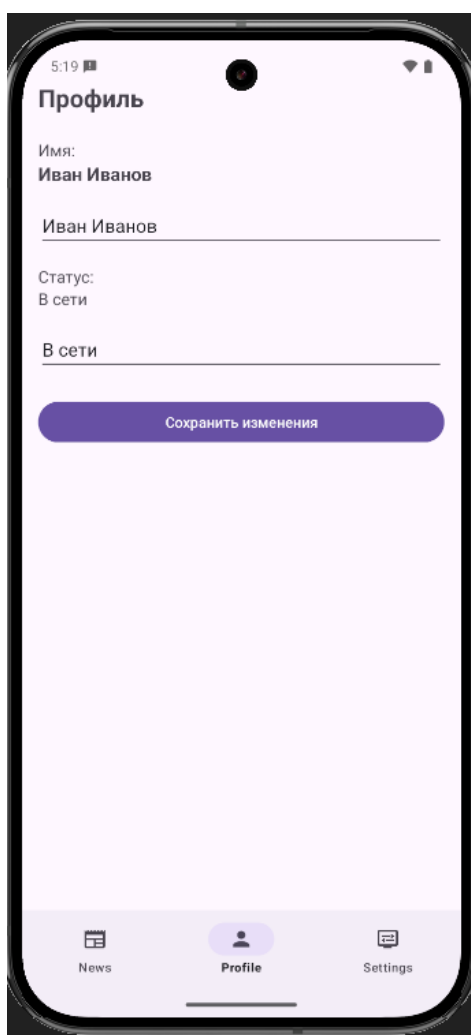


Рисунок 4 - Экран профиля

4 Экран настроек

Пользовательский интерфейс экрана настроек содержит минимальный набор элементов, необходимый для выполнения поставленных требований. Основным элементом управления является переключатель (Switch), отвечающий за выбор темы приложения — светлой или тёмной.

Для хранения состояния настроек используется отдельная ViewModel - SettingsViewModel. Она инкапсулирует текущее состояние темы приложения и предоставляет его в виде объекта LiveData. Это обеспечивает реактивное обновление интерфейса при изменении состояния переключателя.

Внешний вид экрана настроек показан на рисунке 5.

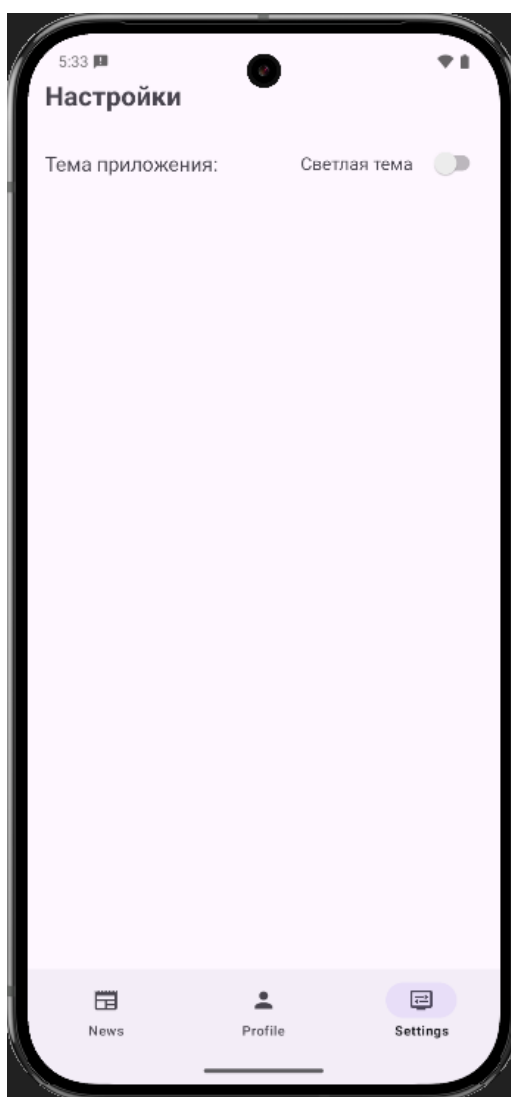


Рисунок 5 - Экран настроек

5 Загрузка данных из сети

Сетевой слой приложения отвечает за выполнение HTTP-запросов к удаленному серверу и получение данных в формате JSON. Его основными задачами являются:

- установление соединения с сервером;
- выполнение запросов на получение списка сообщений;
- обработка ответов сервера;
- преобразование полученных данных в модели, используемые внутри приложения;
- передача данных в репозиторий для дальнейшей обработки и хранения.

Выделение сетевого слоя в отдельный компонент позволяет изолировать сетевую логику от пользовательского интерфейса и ViewModel, что повышает надежность и расширяемость приложения.

Для реализации сетевого взаимодействия в проекте используется библиотека Retrofit. Retrofit представляет собой высокоуровневый HTTP-клиент, широко применяемый в Android-разработке для работы с REST API. Retrofit позволяет существенно сократить объем шаблонного кода и повысить читаемость реализации сетевого слоя.

Все сетевые операции выполняются асинхронно с использованием Kotlin Coroutines. Это является обязательным требованием Android-платформы, так как выполнение сетевых запросов в главном потоке может привести к блокировке интерфейса и аварийному завершению приложения.

Для загрузки сообщений используется публичный API jsonplaceholder [6].

Схема работы с данными из сети показана на рисунке 6.

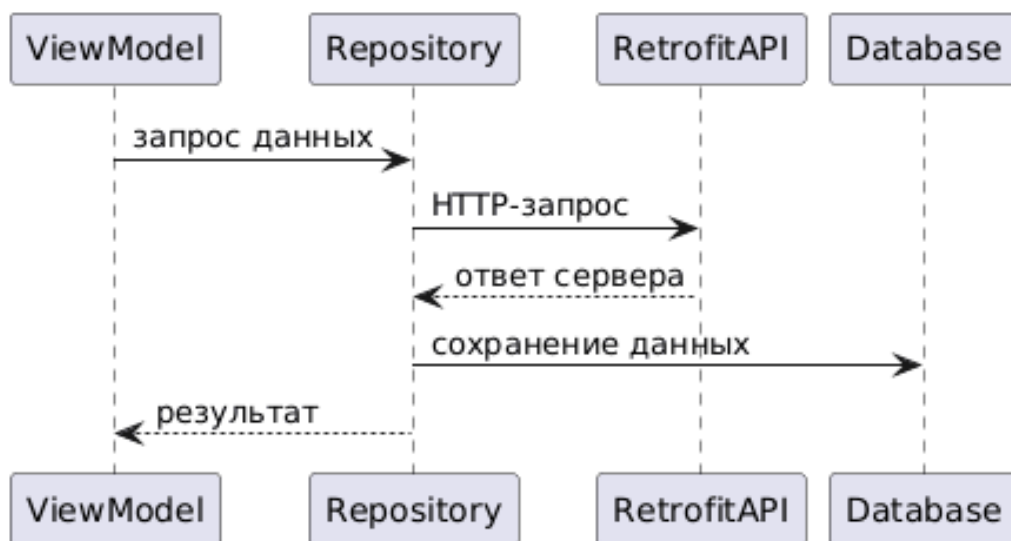


Рисунок 6 - Работа с данными из сети

6 Локальное хранилище данных

6.1 Общее описание

Одним из важнейших требований к современным мобильным приложениям является возможность корректной работы при нестабильном или отсутствующем интернет-соединении. Для решения данной задачи в разрабатываемом Android-приложении «Мессенджер» реализовано локальное хранение данных с использованием библиотеки Room [1].

Локальное хранилище данных в приложении выполняет следующие функции:

- сохранение сообщений, полученных из сети;
- предоставление данных при отсутствии интернет-соединения;
- уменьшение количества сетевых запросов;
- повышение производительности приложения;
- обеспечение целостности и структурированности данных.

6.2 Использование Room

Для работы с локальной базой данных используется библиотека Room. Room представляет собой абстракцию над SQLite и предоставляет удобный и типобезопасный API для работы с базами данных в Android-приложениях. Выбор Room обусловлен её официальной поддержкой со стороны Google и соответствием современным стандартам Android-разработки.

В рамках приложения создана локальная база данных, содержащая таблицу сообщений. Каждое сообщение хранится в виде отдельной записи и включает в себя основные поля, необходимые для отображения и взаимодействия с пользователем.

Диаграмма структуры данных показана на рисунке 7.

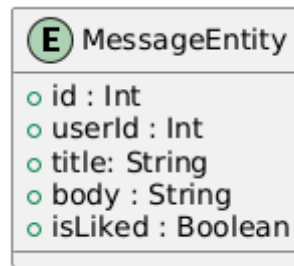


Рисунок 7 - Структура данных

6.3 Работа с данными

Для доступа к данным используется интерфейс DAO (Data Access Object). DAO определяет набор методов, с помощью которых осуществляется взаимодействие с базой данных. Использование DAO позволяет изолировать SQL-логику от остального кода приложения и повысить читаемость и сопровождаемость проекта.

Все операции с базой данных выполняются асинхронно с использованием Kotlin Coroutines. Это предотвращает блокировку главного потока и обеспечивает плавную работу пользовательского интерфейса.

Room не используется напрямую во ViewModel или пользовательском интерфейсе. Все обращения к базе данных выполняются через репозиторий, который объединяет работу с сетевым и локальным источниками данных.

Схема взаимодействия с репозиторием показана на рисунке 8.

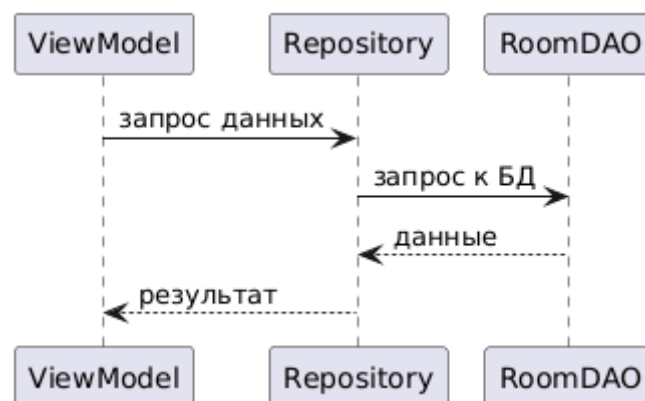


Рисунок 8 - Взаимодействие с репозиторием

При отсутствии интернет-соединения репозиторий автоматически переключается на использование локальной базы данных. Пользователь при этом продолжает видеть список сообщений, сохраненных ранее.

Использование Room обеспечивает контроль целостности данных и предотвращает некорректные операции с базой данных. В случае ошибок данные не теряются, а приложение продолжает функционировать в стабильном режиме.

7 Отображение списка сообщений

7.1 Использование RecyclerView

Для отображения списка сообщений используется компонент RecyclerView, предназначенный для эффективной работы с большими наборами данных. RecyclerView обеспечивает повторное использование элементов интерфейса, что положительно сказывается на производительности приложения.

RecyclerView используется во фрагменте ленты сообщений и получает данные из ViewModel, которая, в свою очередь, взаимодействует с репозиторием.

7.2 Специализированный адаптер

Для управления отображением элементов списка реализован кастомный RecyclerView.Adapter. Каждый элемент списка представлен отдельным ViewHolder и содержит следующие компоненты:

- аватар пользователя;
- имя автора сообщения;
- текст сообщения;
- иконку лайка.

При разработке интерфейса применялись рекомендации Material Design. В частности, использовались следующие компоненты:

- CardView для оформления каждого сообщения;
- AppBarLayout для верхней панели приложения;
- FloatingActionButton для выполнения ключевых действий (обновление данных).

Внешний вид ленты сообщений показан на рисунке 9.

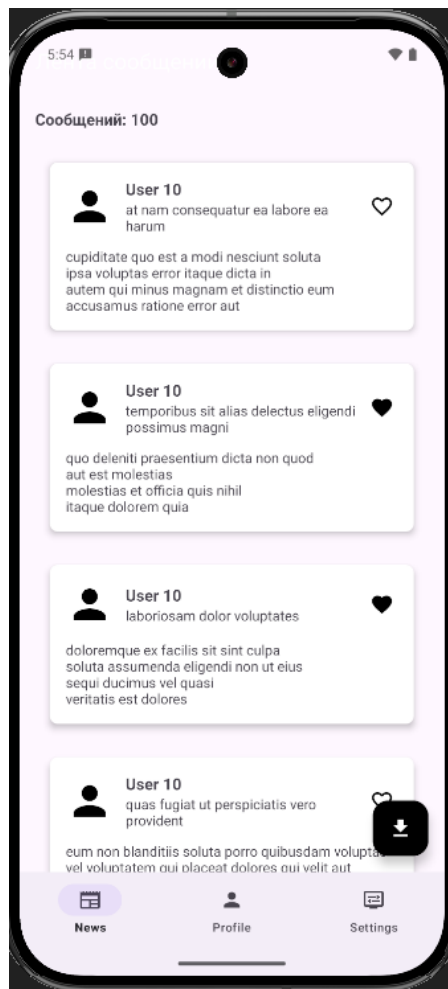


Рисунок 9 - Лента сообщений

Пользователь может отметить сообщение как понравившееся. Состояние лайка сохраняется в базе данных и корректно восстанавливается при повторной загрузке данных.

8 Фоновая синхронизация

Для обеспечения актуальности данных в приложении реализована фоновая синхронизация сообщений. Это позволяет обновлять информацию даже в тех случаях, когда пользователь не взаимодействует напрямую с приложением.

Для фоновой синхронизации создан отдельный Worker-класс, который:

- выполняет сетевой запрос через репозиторий;
- сохраняет обновленные данные в локальную базу;
- сообщает о результате выполнения задачи.

Все операции внутри Worker выполняются асинхронно с использованием корутин. Его схема работы показана на рисунке 10.

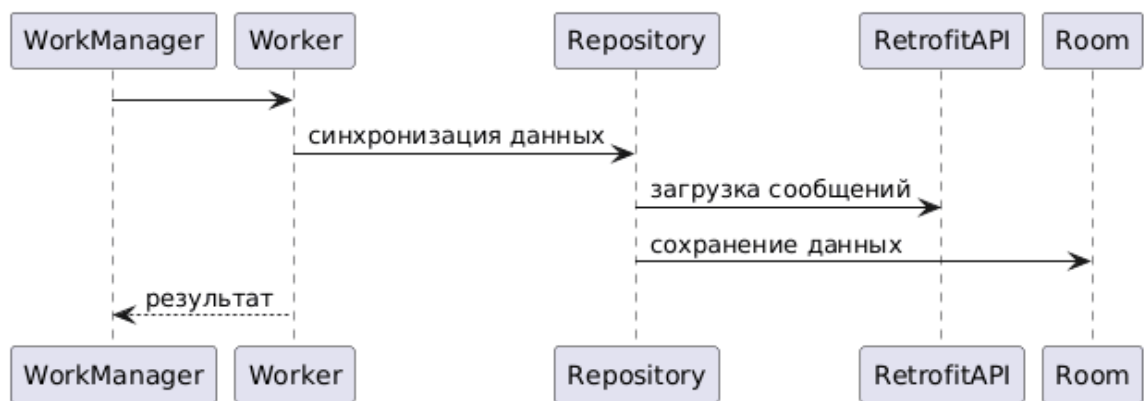


Рисунок 10 - Схема работы Worker

9 Уведомления и разрешения

Для информирования пользователя о результатах фоновой синхронизации в приложении реализована система уведомлений. Также предусмотрена обработка необходимых разрешений операционной системы Android.

9.1 Уведомления

При успешной фоновой синхронизации пользователю отображается системное уведомление с сообщением о получении новых данных. Это позволяет своевременно информировать пользователя без необходимости постоянного взаимодействия с приложением.

Уведомления инициируются из Worker-класса после успешного завершения синхронизации. Такой подход обеспечивает корректную работу уведомлений независимо от состояния пользовательского интерфейса.

Уведомление показано на рисунке 11.

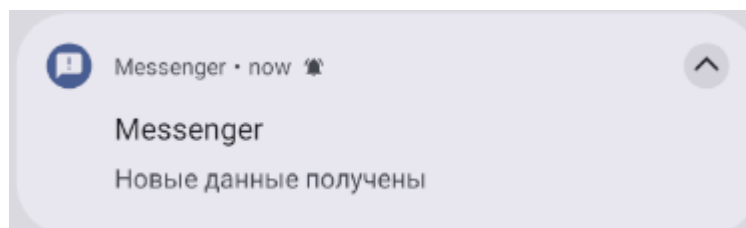


Рисунок 11 - Уведомление о получении новых данных

9.2 Разрешения

При первом запуске приложения запрашиваются необходимые системные разрешения, включая разрешение на показ уведомлений. Запрос разрешений осуществляется в соответствии с рекомендациями Android и сопровождается пояснением для пользователя.

Запрос разрешения показан на рисунке 12.

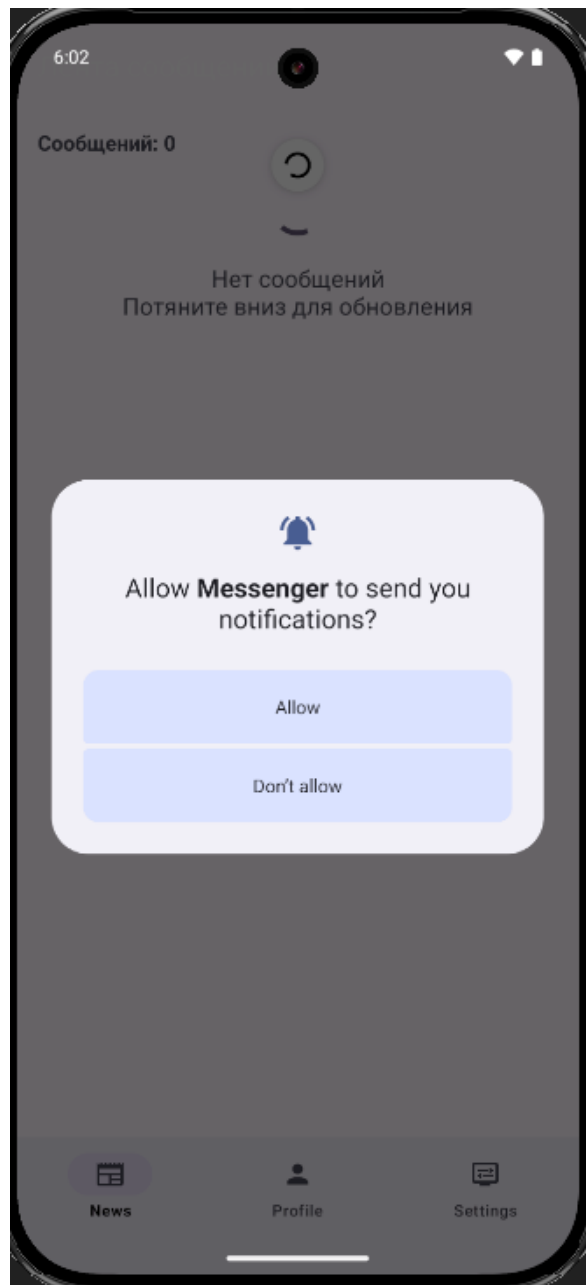


Рисунок 12 - Запрос разрешения

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной работы было разработано Android-приложение «Messenger», соответствующее всем поставленным функциональным и нефункциональным требованиям. В процессе разработки были применены современные архитектурные подходы, инструменты и библиотеки Android-платформы.

Реализованное приложение демонстрирует:

- использование архитектуры MVVM;
- работу с сетевыми запросами и локальным хранилищем данных;
- поддержку офлайн-режима;
- реактивное обновление пользовательского интерфейса;
- фоновую синхронизацию и систему уведомлений.

Полученные в ходе работы навыки и знания могут быть использованы при разработке более сложных мобильных приложений и послужат основой для дальнейшего изучения Android-разработки.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) Android Developers. Room Persistence Library. - URL: <https://developer.android.com/training/data-storage/room> (дата обращения: 21.01.2026).
- 2) Android Developers. Navigation component. - URL: <https://developer.android.com/guide/navigation> (дата обращения: 22.01.2026).
- 3) Kotlin Documentation. - URL: <https://kotlinlang.org/docs/home.html> (дата обращения: 21.01.2026).
- 4) Square. Retrofit. - URL: <https://square.github.io/retrofit/> (дата обращения: 22.01.2026).
- 5) Material Design 3. - URL: <https://m3.material.io/> (дата обращения: 20.01.2026).
- 6) JSONPlaceholder. Free fake API for testing and prototyping. - URL: <https://jsonplaceholder.typicode.com/> (дата обращения: 20.01.2026).